Data Parallelism

DSC 102: Systems for Scalable Analytics

Haojian Jin

UC San Diego

Where we are

- **L4:** split the **tasks** across workers (task parallelism)
- **L5:** split the **data** across the memory hierarchy of one machine (scalable data access)
- **Today:** split the data across **machines**: this is where parallelism and scalability finally meet

Data parallelism: sharding, BSP, distributed filesystems, data-parallel operations, and hybrid parallelism.

Five questions for today

1. What is **data parallelism**, and where does it fit?
2. How do we **partition** data across nodes?
3. How do nodes **coordinate**: architectures and protocols?
4. How do the core **data science operations** run data-parallel?
5. What do **optimizations** and **hybrid parallelism** buy?

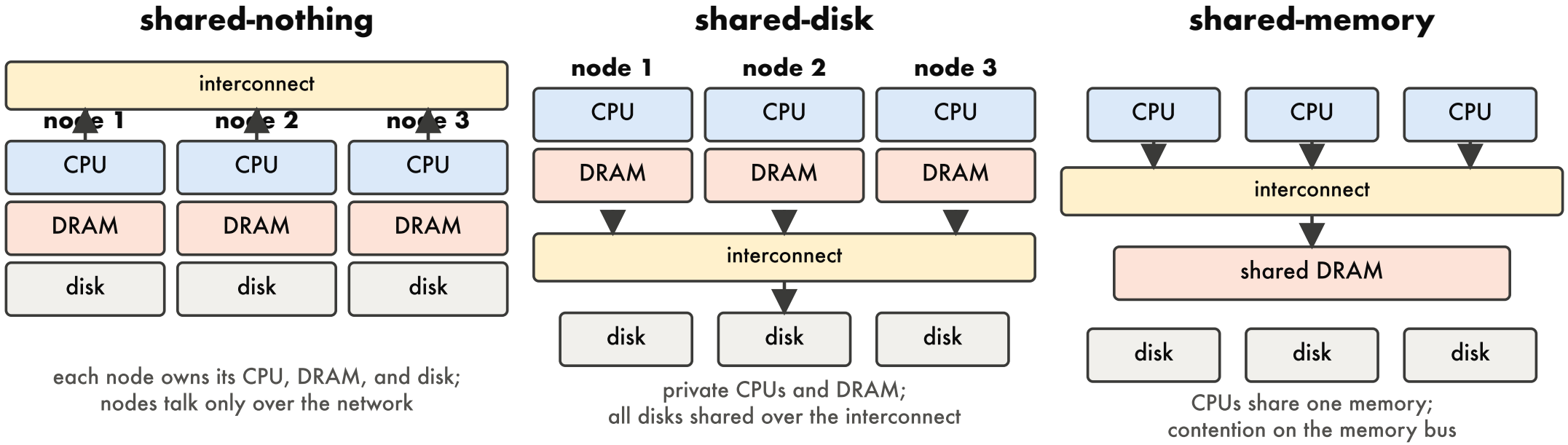
Q1 · Introducing data parallelism

Basic idea of scalability (L5): split a data file, virtually or physically, and stage reads and writes of its pages between disk and DRAM.

Data parallelism: partition a large data file **physically across nodes or workers**; within a worker, process it DRAM-based or disk-based as before.

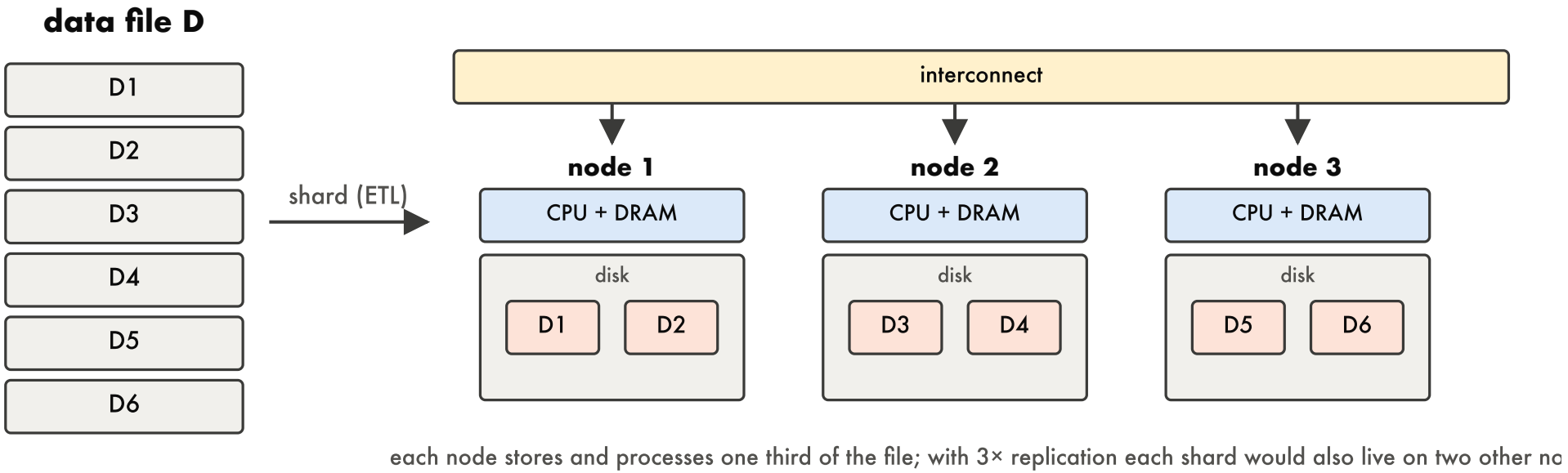
- The most common approach to marrying parallelism and scalability in data systems
- A generalization of the SIMD and SPMD idea from parallel processors to large-scale data on many workers
- Two flavors: **distributed memory** (the data fits in the cluster's total DRAM) and **distributed disk**

Three paradigms of multi-node parallelism



data parallelism is orthogonal to these, but is almost always paired with shared-nothing

Shared-nothing data parallelism



- Partitioning a file across nodes is **sharding**: one stage of **ETL** (extract, transform, load), the umbrella term for everything done to a file before it is ready to query

Data parallelism in the other paradigms?

Shared-disk cluster

- Every node can read every shard, since all disks are shared
- But all nodes pull from the same disks: **contention** on the storage and its interconnect

Shared-memory cluster

- The shards sit in one DRAM that every CPU can reach
- The memory bus becomes the bottleneck: **contention** again

Shared-nothing scales because the only shared resource is the network, and data parallelism uses it sparingly.

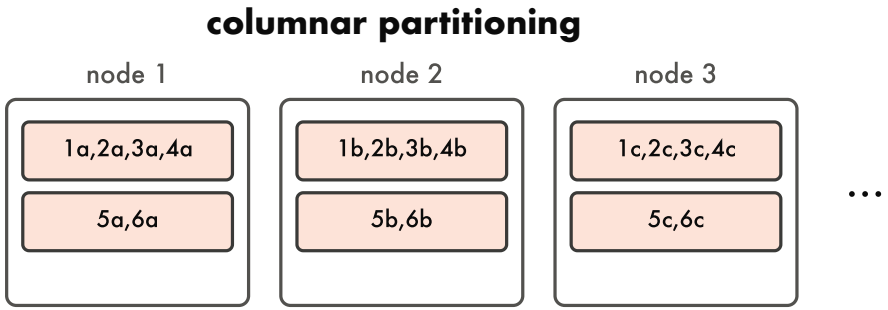
Q2 · Data partitioning strategies

- **Row-wise (horizontal)** partitioning is the most common: sharding
- **Hashing** is most common in practice for relational algebra and SQL; **range** partitioning suits range predicates; all three are often fine for ML workloads (why?)
- **Replication** of each partition across nodes (e.g. 3×) is common: fault tolerance and better parallel performance

Other forms of data partitioning

A	B	C	D
1a	1b	1c	1d
2a	2b	2c	2d
3a	3b	3c	3d
4a	4b	4c	4d
5a	5b	5c	5d
6a	6b	6c	6d

R



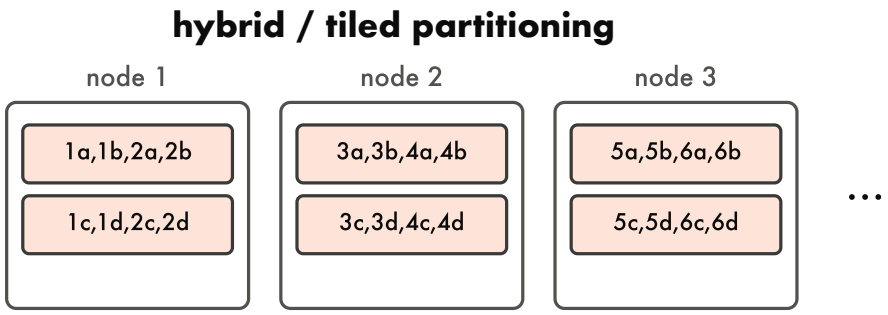
node 1 holds column A's pages, node 2 column B's, ...: a column-store across the cluster

Just like the disk-aware layouts of a single node, a file can be partitioned across workers by **column**, or in **tiles**: see the next slide.

Tiled partitioning across nodes

A	B	C	D
1a	1b	1c	1d
2a	2b	2c	2d
3a	3b	3c	3d
4a	4b	4c	4d
5a	5b	5c	5d
6a	6b	6c	6d

R

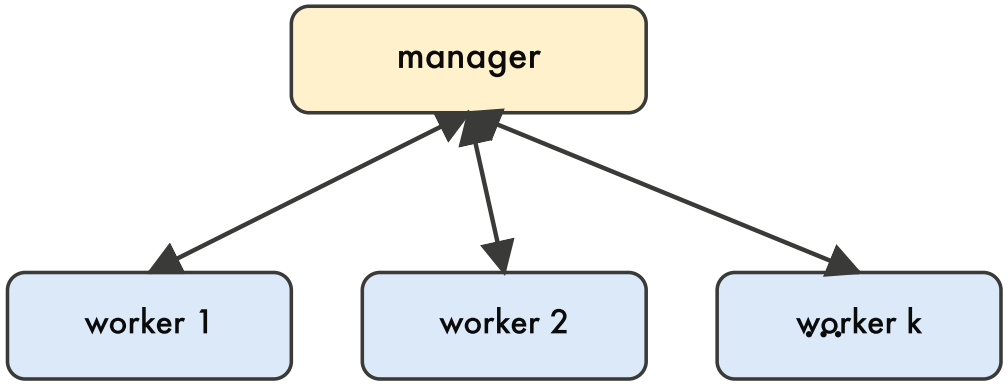


node 1 holds the tiles of rows 1–2, node 2 rows 3–4, ...: a tiled store across the cluster

Layout within a node and partitioning across nodes are independent choices: a row-store can be partitioned by column, and vice versa.

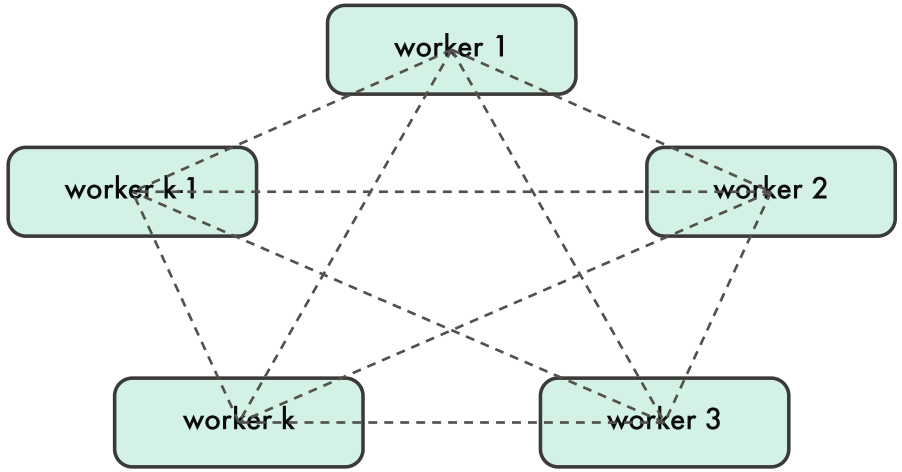
Q3 · Cluster architectures

manager–worker (centralized)



the manager says what to do and when to talk;
Dask, Spark, parallel RDBMSs

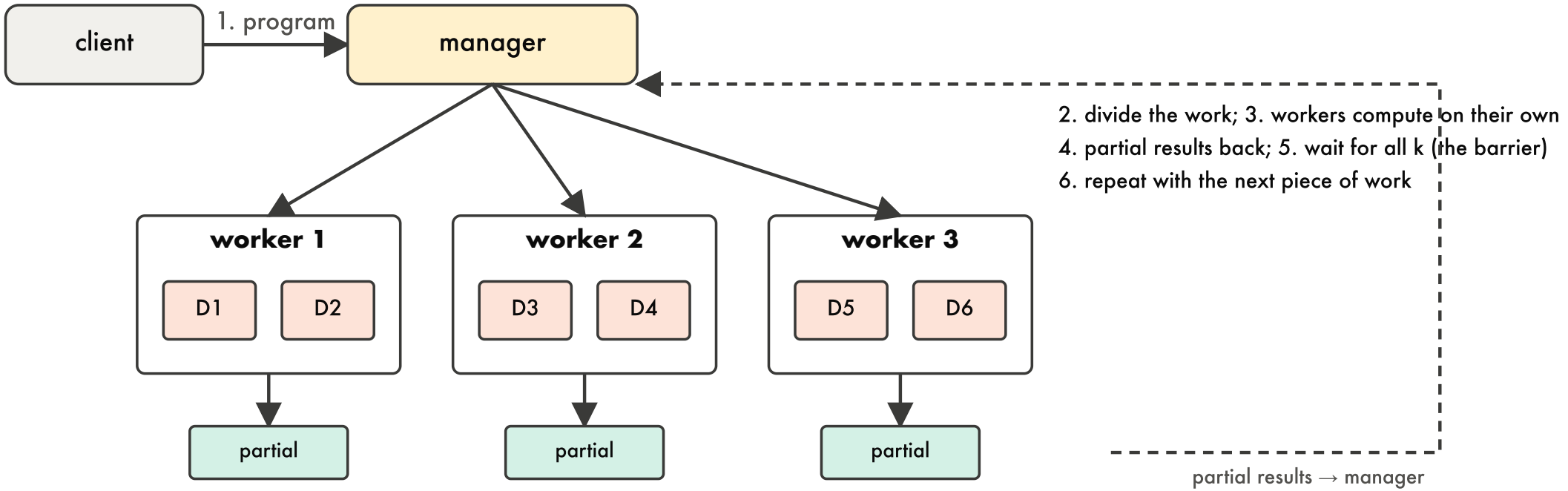
peer-to-peer (decentralized)



no special node; workers talk to each other directly; e.g. Horovod

Q: What is the protocol for the nodes to talk to each other?

Bulk synchronous parallelism (BSP)



sharded data + manager–worker + barrier synchronization = bulk synchronous parallelism; the communication costs

The most common protocol of data parallelism in data systems: parallel RDBMSs, Hadoop, Spark.

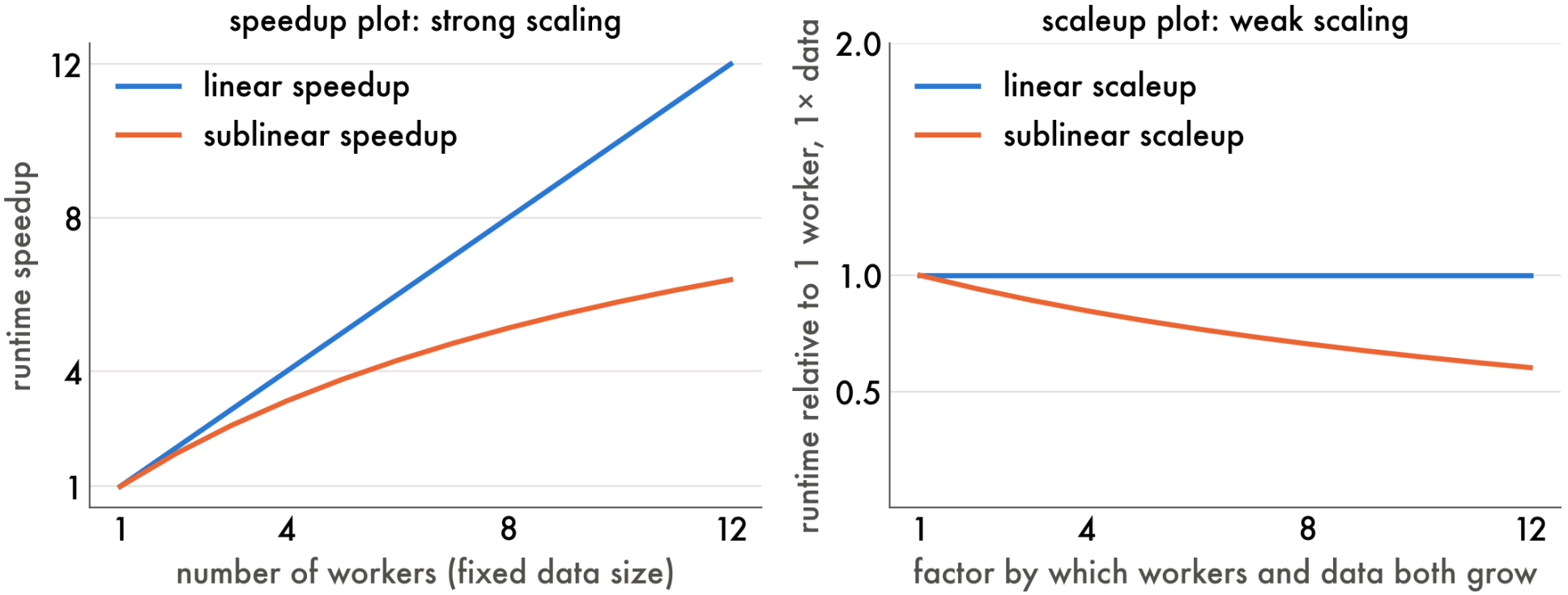
Speedup limits of BSP

$$\text{Speedup} = \frac{\text{completion time with 1 worker}}{\text{completion time with } k \text{ workers}}$$

Cluster overhead factors that hurt the speedup:

- **Per worker:** startup cost, tear-down cost
- **On the manager:** dividing up the work; collecting and unifying the partial results
- **Communication:** manager-to-worker traffic, and worker-to-worker traffic when the manager asks for it
- **Barrier synchronization suffers from stragglers:** skew in shard sizes or in worker capacities means everyone waits for the slowest

Speedup and scaleup, on a cluster



Strong scaling fixes the data and adds workers; **weak scaling** grows both. **Q: Is superlinear speedup ever possible on a cluster?**

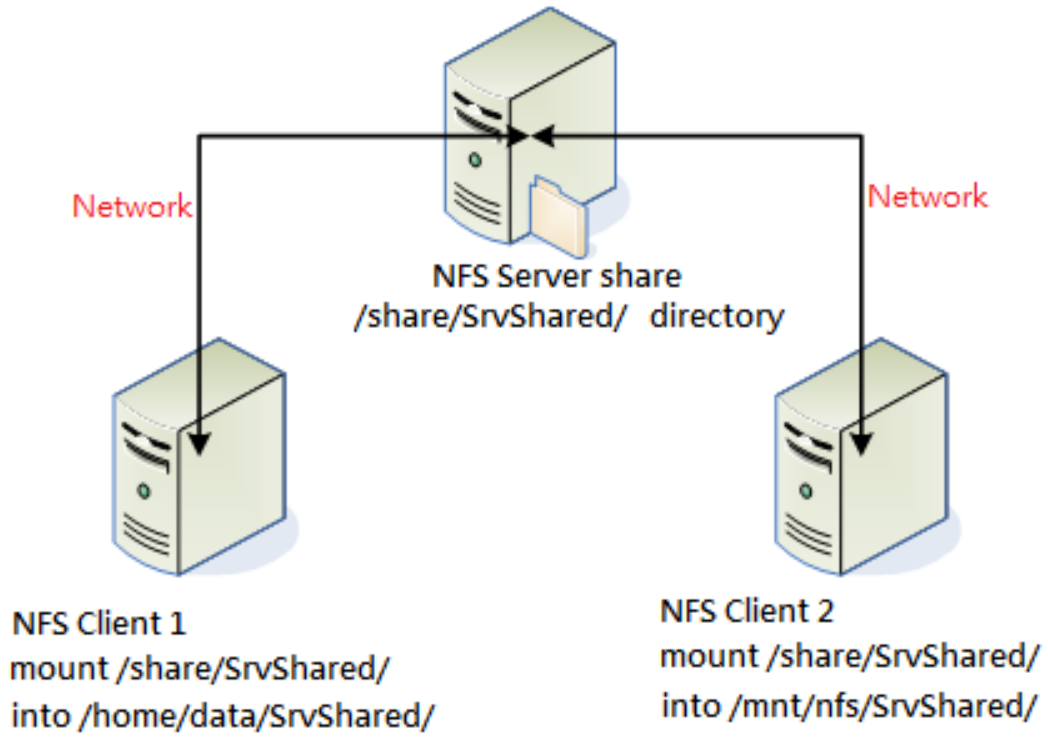
Distributed filesystems

- A **distributed file** generalizes the file to a cluster of networked disks and OSs
- A **distributed filesystem (DFS)** is a cluster-resident layer on top of the local filesystems: nodes manage their data as local files, and DFS APIs give the illusion of **one global file** reachable from any node

Two main variants:

Remote DFS	Files reside elsewhere and are read or written on demand by the workers
In-situ DFS	Files reside on the cluster where the workers run

Network File System (NFS)



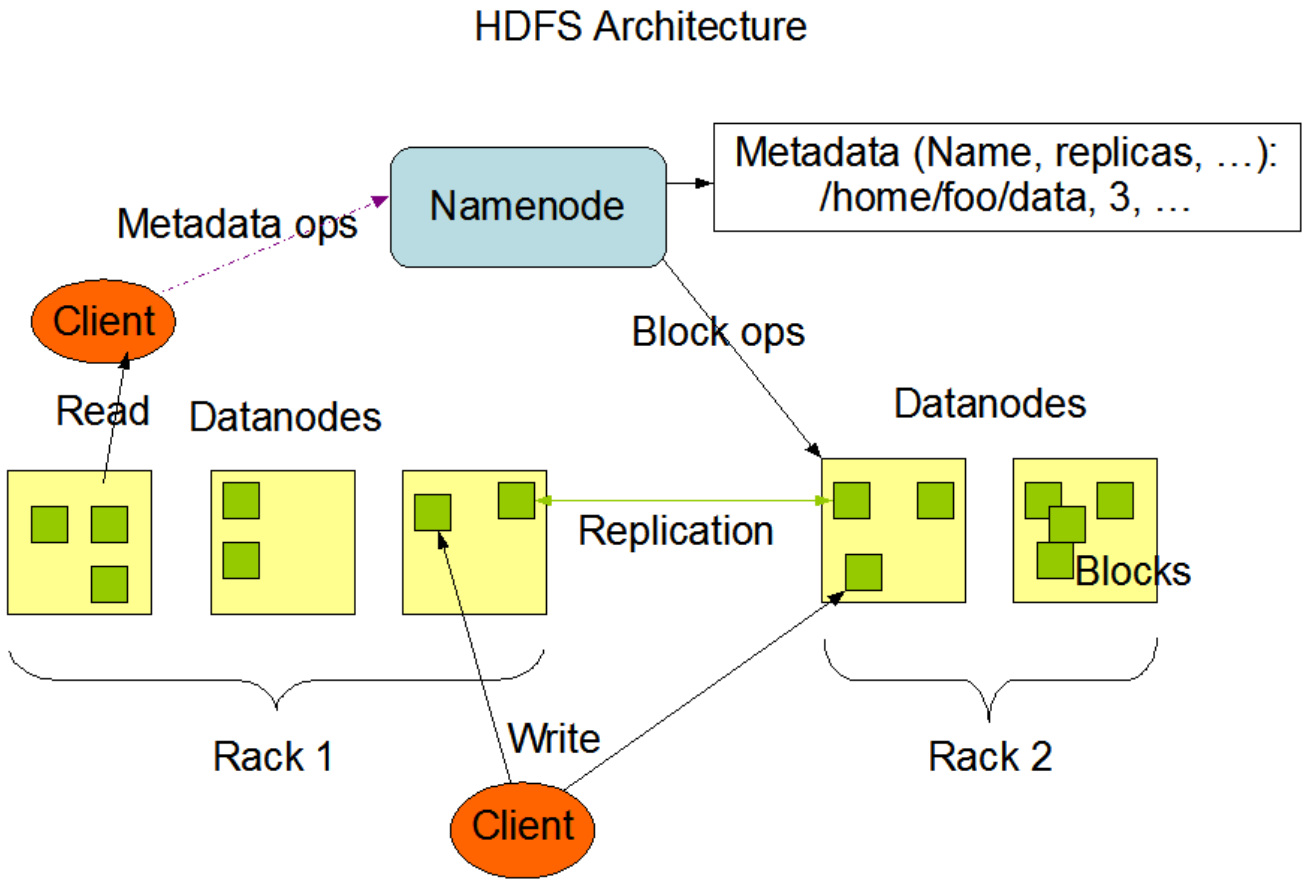
An old **remote DFS** (1980s) with a simple client-server architecture for replicating files over the network.

Main pro: simplicity of setup and use.

Cons:

- Not scalable to very large files
- Full data replication
- High contention for concurrent reads and writes
- A single point of failure

Hadoop Distributed File System (HDFS)

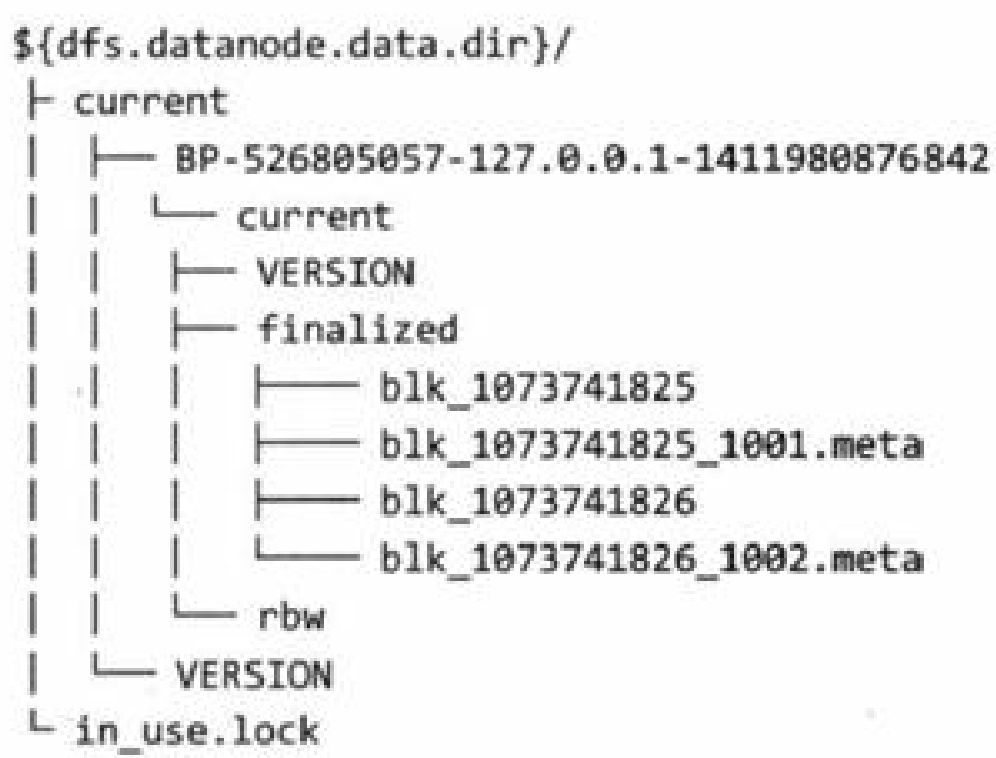


The most popular **in-situ DFS** (late 2000s), part of Hadoop; the open-source spinoff of the **Google File System**.

- Tens of thousands of cheap commodity nodes, petabyte files
- Parallel reads and writes of sharded **blocks**, **replicated** for fault tolerance
- Con: read-only plus batch-append, no fine-grained updates

Ghemawat et al., The Google File System, SOSP 2003 · https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html

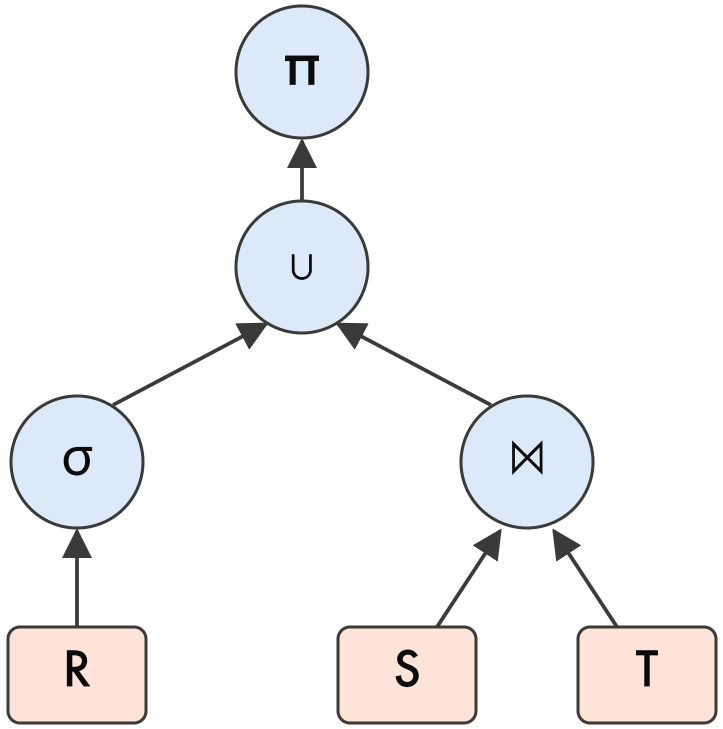
HDFS internals



- The **NameNode**'s roster maps data blocks to **DataNodes** and their IPs
- A distributed file on HDFS is just a **directory** with a file per data block plus metadata files
- The block size and the replication factor are configurable; the defaults are **128 MB** and **3×**

A "file" that is really a directory of blocks on many disks: the illusion is in the API.

Data-parallel dataflows and workflows



$\pi(\sigma(R) \cup (S \bowtie T))$

- **Data-parallel dataflow:** each operator runs data-parallel over sharded inputs
- **Data-parallel workflow:** each vertex is a whole task run data-parallel
- Every extended relational operator has a data-parallel implementation in parallel RDBMSs and Spark

Q: How do we run data science operations data-parallel?

Q4 · Data-parallel data science operations

The same representative operations as L5, now on a sharded file:

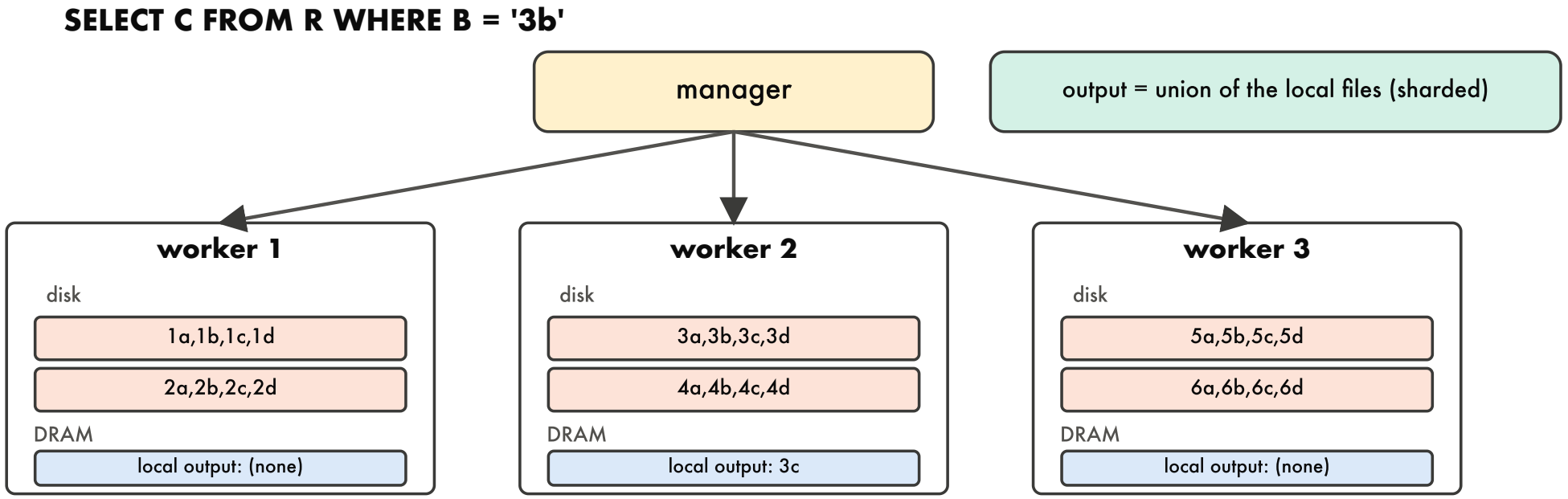
Database systems: select · non-deduplicating project · simple SQL aggregates · GROUP BY aggregates

ML systems: matrix sums and norms · stochastic gradient descent

Basic idea, BSP style: the manager **splits the work** → each worker does **node-local work** on its shard, exactly as in L5 → the manager **unifies the results**.

After ETL, the sharded input file already sits on the cluster's disks.

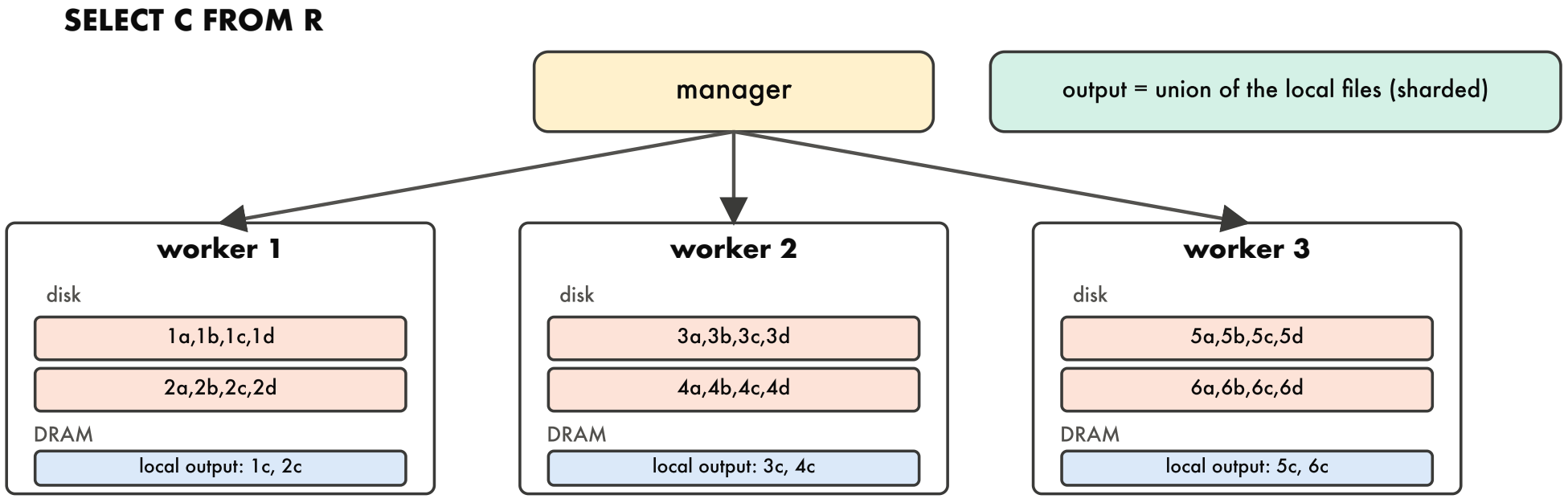
Data-parallel relational select



I/O costs: disk 6 + output · network 0

The manager broadcasts the query; each worker runs the L5 filescan on its own shard and writes a **local** output file; the union of the local files is the global output, **also sharded**.

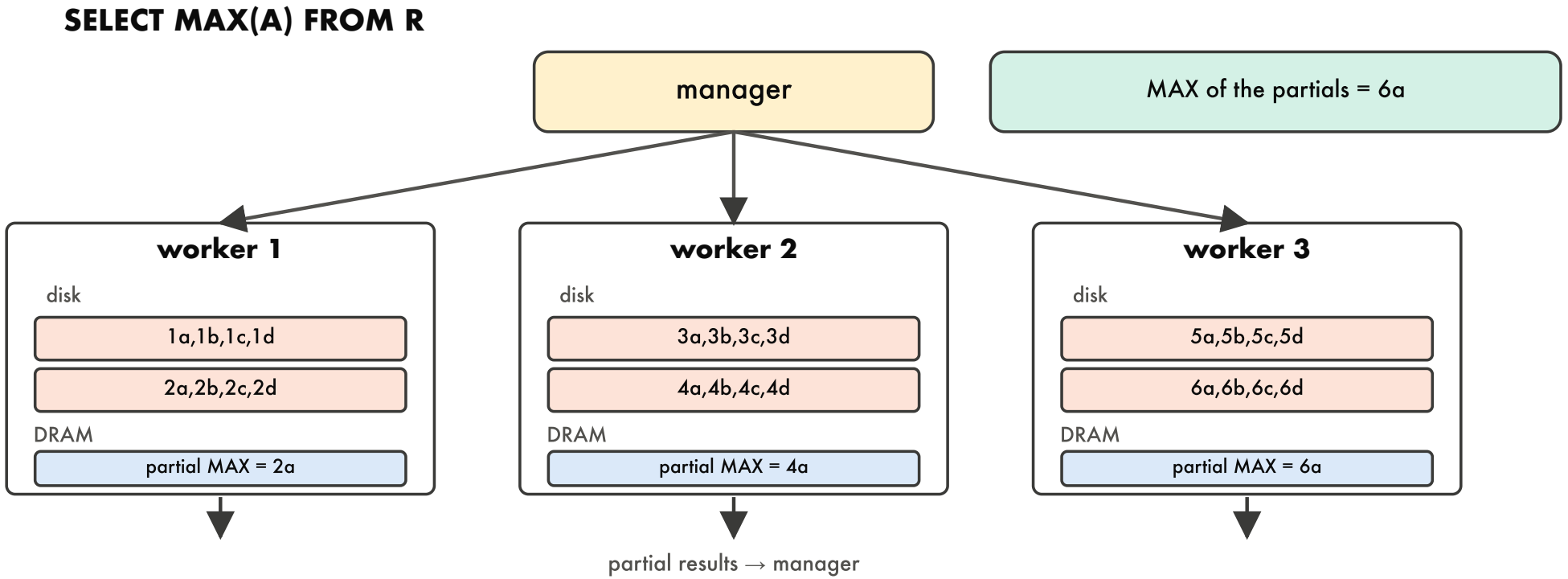
Data-parallel non-deduplicating project



I/O costs: disk 6 + output · network 0

Identical pattern: node-local project, local output files, union at the manager. **No network traffic for the data itself.**

Data-parallel simple aggregates



I/O costs: disk 6 + output · network 3 (one datum per worker)

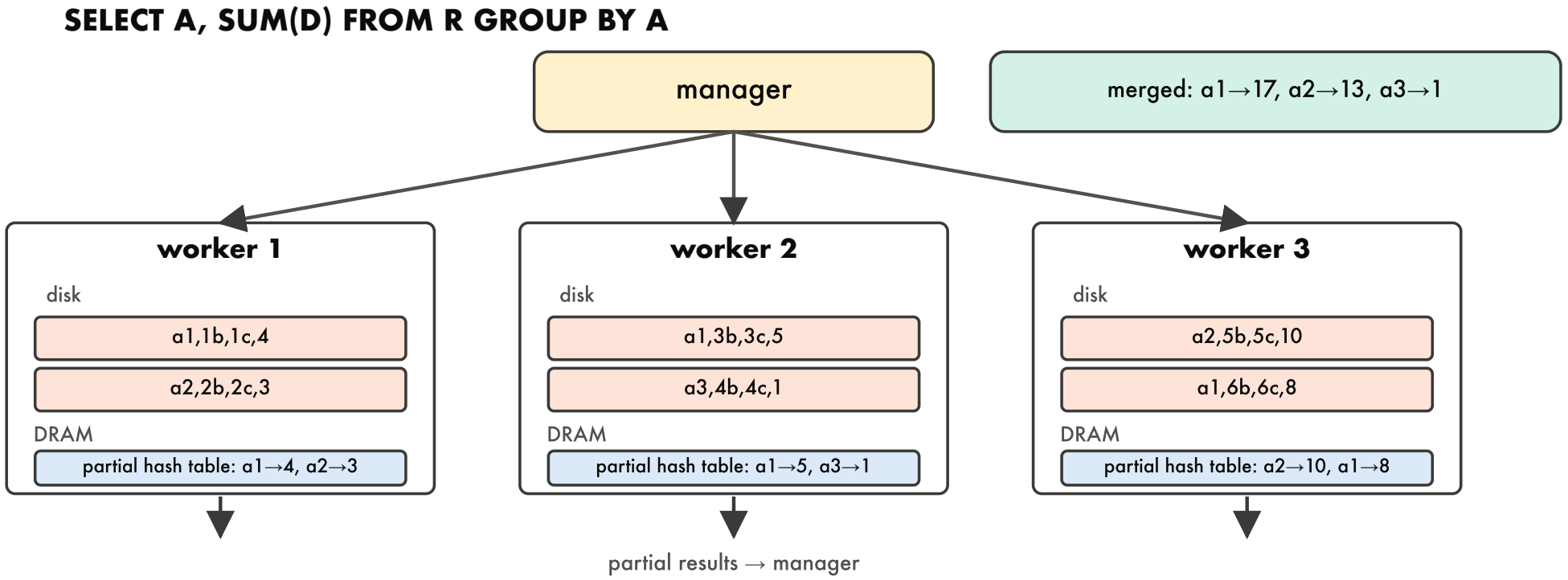
Each worker computes a **partial aggregate** on its shard and sends it to the manager, which unifies the partials according to the operator's semantics.

Which aggregates split easily?

Q: Are all SQL aggregates easy to split over sharded data? They fall into three types:

Type	A shard sends the manager	Examples
Distributive	exactly **one datum**	MIN, MAX, COUNT, SUM
Algebraic	**O(1)** statistics	AVG (SUM and COUNT), VARIANCE and STDEV (SUM, SUM of squares, COUNT)
Holistic	O(1) is **not enough** in general; larger intermediate state	MEDIAN, MODE, PERCENTILES

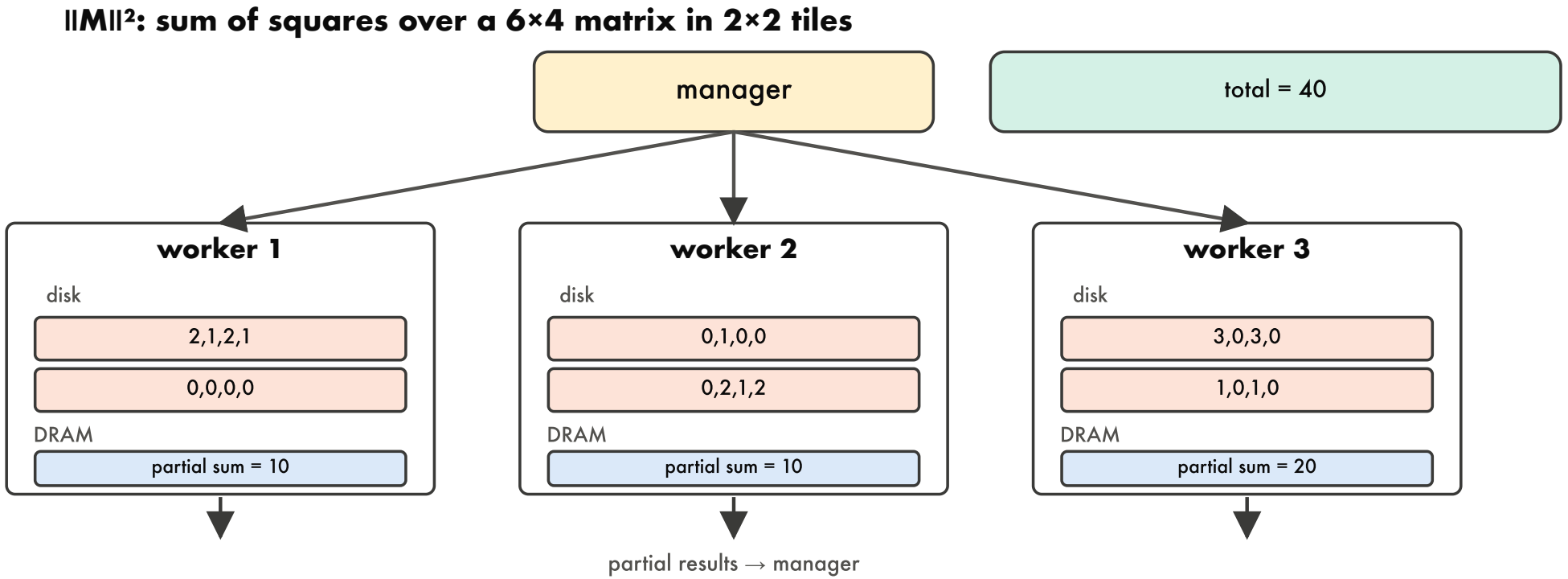
Data-parallel GROUP BY aggregate



I/O costs: disk 6 + output · network 3 partial hash tables · network cost grows with the domain of A

Each worker builds a **partial hash table** on its shard; the manager collects and **merges** them into the global output. **Q: What if the manager's DRAM cannot hold all the hash tables?**

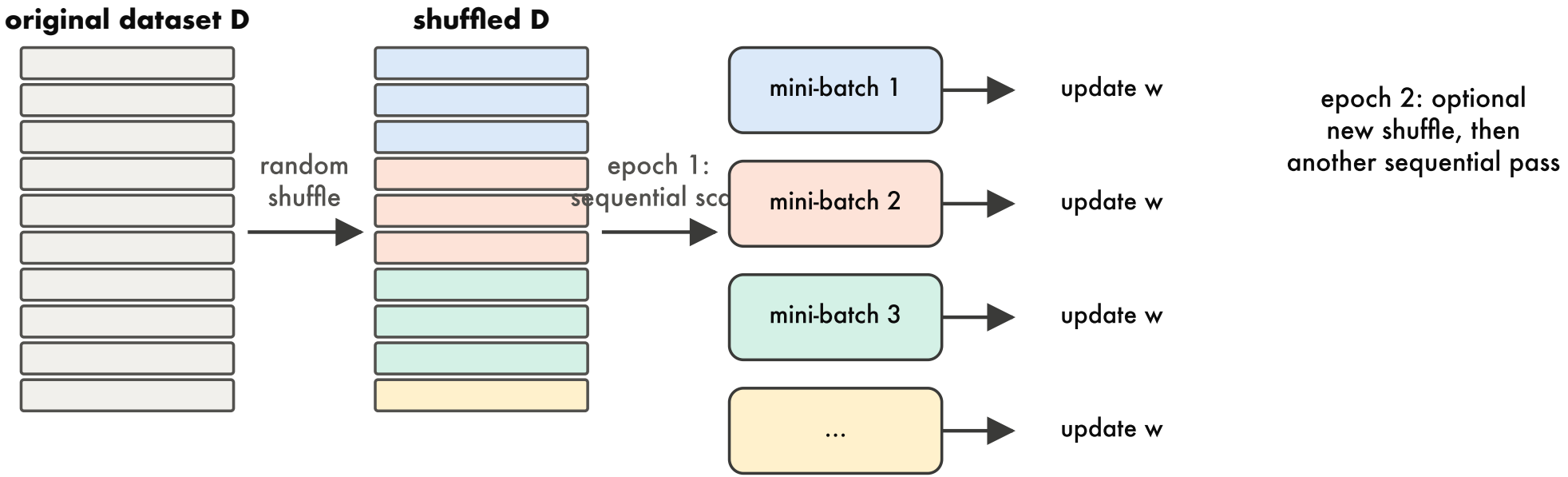
Data-parallel matrix sum and norm



I/O costs: disk 6 · network 3 (one number per worker)

Similar to a data-parallel simple aggregate: with a 2×2 tiled layout and partitioning, each worker sums the squares of its tiles and sends one number.

The access pattern of scalable SGD (recap)



in SQL terms: ORDER BY RAND(), then one filescan per epoch

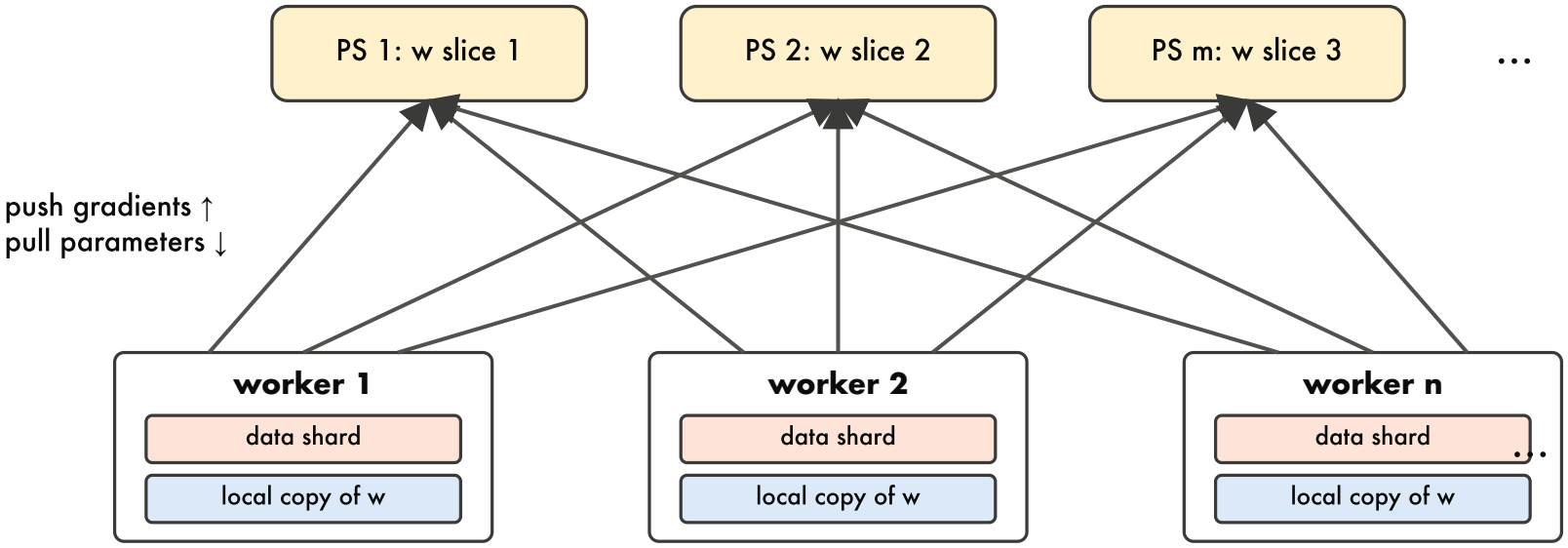
Why SGD is not an algebraic aggregate

An SGD epoch is similar to an SQL aggregate, but also different:

- More complex aggregation state (running information): the **model parameters** w
- **Multiple** mini-batch updates to w within one pass
- A **sequential dependency** across the mini-batches in a pass
- **Not commutative:** different shuffle orders give different results, very unlike relational operators
- The state must be kept **across epochs**, with an optional reshuffle before each one

Not an algebraic aggregate, so hard to parallelize. Q: How do we execute SGD data-parallel at all?

The parameter server for scalable SGD



no barrier: workers and servers never wait for each other; parameters may be stale, but SGD is robust; network cost is high

- A **multi-server manager**, each holding a slice of w; workers **push** gradients and **pull** parameters per mini-batch, or less often, with no synchronization
- Parameters go **stale**, but SGD is **robust**; the price is **high network I/O** (more in CSE 234)

Q5 · Common optimizations in data-parallel systems

Optimization	Idea
Replication	Put a shard on more than one worker: more parallelism possible for execution, and fault tolerance
Caching	Keep as much data as possible in worker DRAM and on worker disk
Asynchrony	Drop the barrier; rare in DB systems, common in ML systems (the parameter server)
Approximation	Carefully exploit data subsampling

Using **ML for data placement, caching, and tiered storage** across the memory hierarchy is a hot topic in the "ML for systems" world.

Hybrid parallelism

Task and data parallelism have complementary pros and cons:

Task parallelism

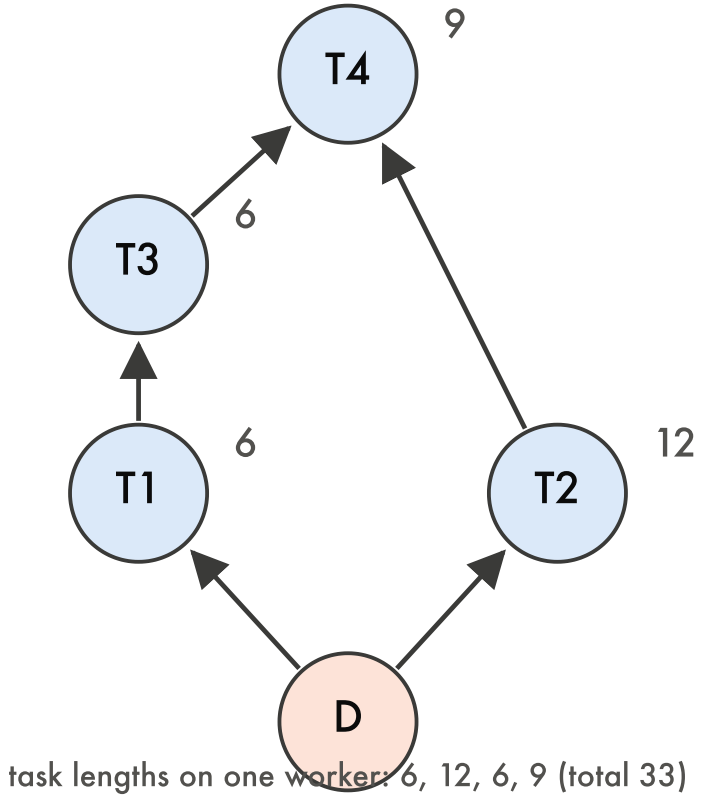
- Wastes memory and storage on **replication**, and network on remote reads
- But **easy** to implement

Data parallelism

- **Painful** to implement at the operator level
- But scales without wasting memory or storage; more network cost

Q: Can we get the best of both worlds? Yes: often we can run task parallelism **on sharded data**. Different SQL queries, or different ML training runs, over the same sharded file: "multi-query execution" in the DB world.

Task parallelism versus BSP: an example



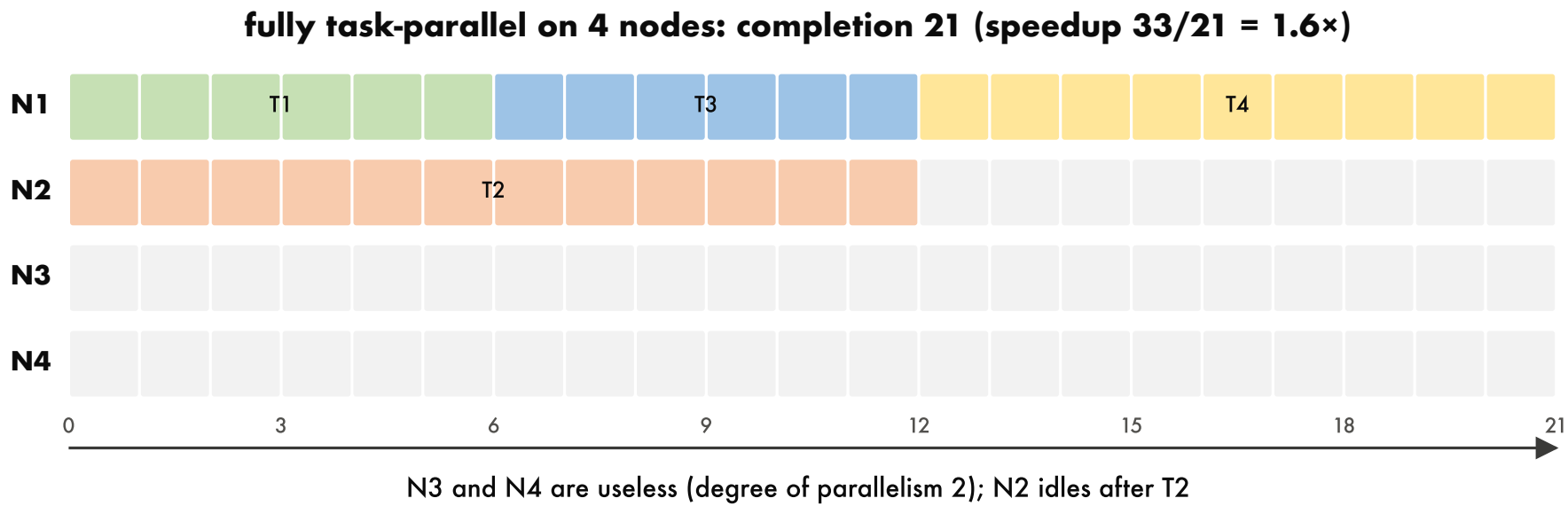
task lengths on one worker: 6, 12, 6, 9 (total 33)

Four tasks on **4 nodes**.

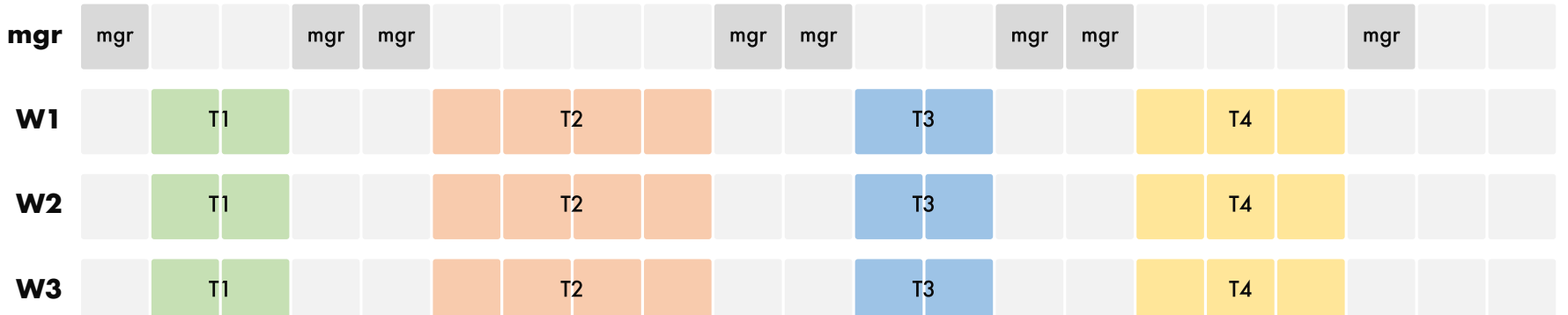
- Run **fully task-parallel**: one task per node
- Or **fully BSP**: one manager and three workers; suppose each task gets a perfect linear speedup on its useful work, and the manager spends **1 unit** before and after each task

Which finishes first?

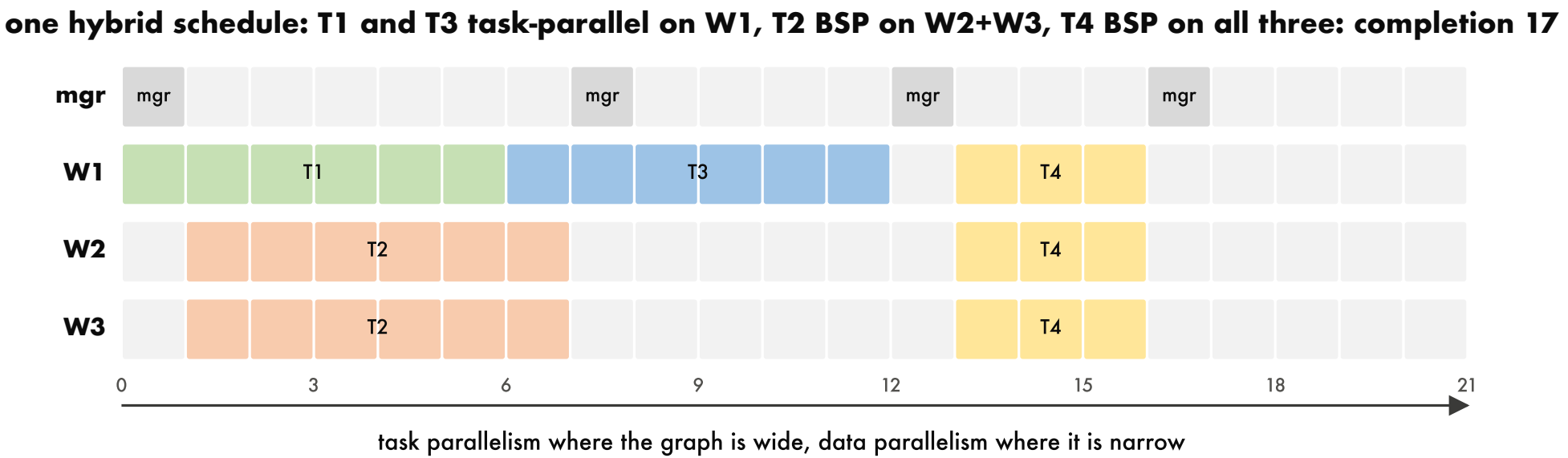
Fully task-parallel versus fully BSP



BSP data-parallel: 1 manager + 3 workers, 1 unit of manager overhead before and after each task: completic

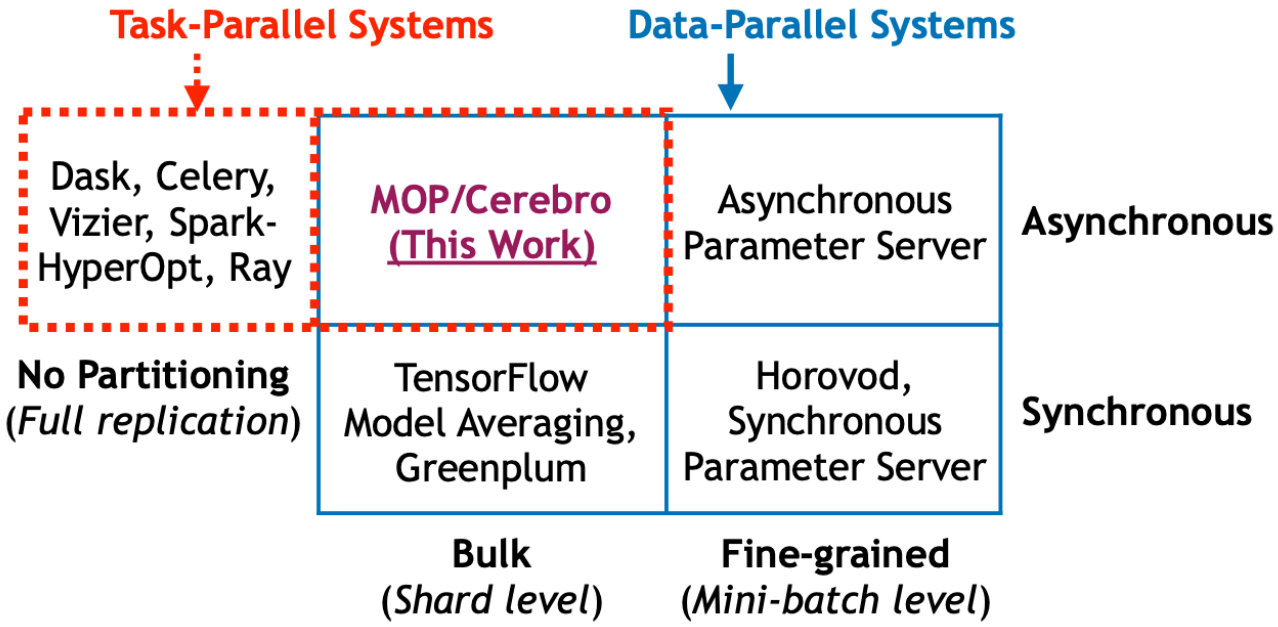


One hybrid schedule



- Fully task-parallel: **21**; fully BSP: **19**; hybrid: **17**
- Most scalable systems today support only full task parallelism (Dask) or full data parallelism (RDBMSs): the software complexity of hybrids is high
- Some RDBMSs exploit hybrid parallelism internally for relational dataflows; Spark is beginning to support task parallelism too

Hybrid parallelism in research: Cerebro



A recent example: **Cerebro** for parallel deep learning model selection on clusters.

- The first known form of **bulk asynchronous parallelism**: shards stay put, models hop between workers
- **Resource-optimal** when compute, memory and storage, and network are considered holistically

CSE 234 covers Cerebro and model selection systems.

Nakandala, Zhang, Kumar. Cerebro: A Data System for Optimized Deep Learning Model Selection, VLDB 2020 / CIDR 2021. https://adalabucsd.github.io/cerebro.html

Review questions

1. To which multi-node paradigm does data parallelism apply: shared-nothing, shared-memory, or shared-disk?
2. What are the two most common cluster communication architectures in parallel data systems?
3. Can columnar partitioning be combined with a row-store? Vice versa?
4. What exactly is the "synchrony" in BSP?
5. Name two sources of overhead in data-parallel systems that lead to sublinear speedup.
6. Name two SQL aggregates that are not algebraic. Why is SGD not parallelizable the way they are?
7. Why does the parameter server have high communication costs? Name one pro and one con of BSP over task parallelism.

Key takeaways

1. **Shard the data and bring the program to it:** data parallelism on shared-nothing clusters is how parallelism and scalability meet
2. **BSP** is the workhorse protocol: node-local work, partial results, a barrier; the barrier and the communication are its cost
3. **Distributive and algebraic aggregates** finish in one round with k numbers on the network; **GROUP BY** and **SGD** need more care
4. **HDFS** turns a cluster of cheap disks into one replicated, sharded file
5. **Hybrid parallelism** is where the remaining speedup lives, and where the software complexity is